

W09D03 — Docker Build & Run

ML Engineer Journey · Phase 3 — Docker & Clean Architecture · 6 May 2026

1. Analogi & Konsep Inti

Bayangkan Dockerfile adalah **resep kue**. Resep itu sendiri tidak bisa dimakan — ia hanya instruksi. Ketika kamu mengikuti resep di dapur dan kue keluar dari oven, itulah **Image** — hasil jadi yang immutable, bisa disimpan dan dibagikan. Ketika kamu memotong kue itu dan mulai makan, itulah **Container**. Setiap orang yang makan dari kue yang sama mendapat pengalaman identik. Dan kalau kuenya habis (container di-stop), kamu tinggal potong lagi dari kue yang sama (run lagi dari image yang sama).

docker build = proses memanggang kue dari resep.

docker run = memotong dan menyajikan kue tersebut.

Konsep	Perintah	Hasil	Sifat
Dockerfile	—	Instruksi build	Teks, bisa diedit
Image	docker build	Snapshot siap pakai	Immutable, read-only
Container	docker run	Instance yang berjalan	Mutable, ephemeral writable layer

2. Layer System — Hubungan dengan W09D01 & D02

Setiap instruksi di Dockerfile menghasilkan satu layer baru yang read-only. Inilah layer system yang kamu pelajari di D01. Ketika **docker build** berjalan, Docker menumpuk layer-layer ini membentuk Image. Ketika **docker run** dijalankan, Docker menambahkan satu **writable container layer** di atasnya — layer inilah yang ephemeral (hilang saat container di-stop).

Urutan layer dari bawah ke atas di dalam container:

Layer	Asal	Sifat
Base OS (python:3.11-slim)	FROM instruction	Read-only
pip install dependencies	RUN instruction	Read-only
App code	COPY instruction	Read-only
Container layer	docker run	Writable, hilang saat stop

Layer caching: Docker re-use layer yang tidak berubah. Inilah kenapa urutan instruksi di Dockerfile sangat penting — COPY requirements.txt sebelum COPY kode supaya layer pip install tidak rebuild ulang setiap kali kode berubah.

3. docker build — Anatomy

Perintah lengkap untuk build image:

```
docker build -t risk-scoring-api:v1 .
```

Bagian	Arti
docker build	Perintah untuk membangun image dari Dockerfile
-t risk-scoring-api:v1	Tag image dengan format nama:versi
.	Build context = direktori saat ini (semua file dikirim ke Docker daemon)

Yang terjadi saat build: Docker baca .dockerignore → kirim context ke daemon → baca Dockerfile baris per baris → tiap instruksi hasilkan layer → layer tidak berubah diambil dari cache → image tersimpan di local registry.

4. docker run — Anatomy Lengkap

```
docker run -d -p 8000:8000 --name risk-api risk-scoring-api:v1
```

Flag	Arti	Kenapa Penting
-d	Detached mode — run di background	Tanpa ini terminal kamu 'ditahan' oleh container
-p 8000:8000	Port mapping: host:container	Traffic ke localhost:8000 diteruskan ke port 8000 di dalam container
--name risk-api	Beri nama pada container	Tanpa ini Docker assign nama random seperti clever_darwin
risk-scoring-api:v1	Image yang digunakan	Nama:tag dari hasil docker build

Hubungan dengan 0.0.0.0: FastAPI harus listen ke 0.0.0.0 (bukan 127.0.0.1) di dalam container supaya port mapping -p bisa bekerja. 0.0.0.0 artinya 'terima dari semua network interface' — bukan berarti terbuka ke internet.

5. uvicorn X:Y — Aturan Path yang Wajib Dipahami

X adalah Python module path (mengikuti aturan import Python, bukan path filesystem). **Y** adalah nama variabel FastAPI instance. Pemisahannya adalah titik dua (:), bukan slash atau dot.

Cara mudah menentukannya: tanya dirimu, 'Kalau aku mau import FastAPI instance-ku dari terminal, aku tulis apa?' Jawaban itu adalah nilai X:Y-mu.

Struktur Project	WORKDIR	CMD uvicorn
main.py ada di root project/	/app	main:app
main.py ada di project/app/	/app	app.main:app
server.py ada di project/src/	/app	src.server:api
app.py ada di root, instance bernama api	/app	app:api

Syarat penting: Setiap folder yang jadi Python package harus punya `__init__.py`. Tanpa file ini, Python tidak menganggap folder itu sebagai package dan import akan gagal.

6. Debug Workflow — Container Langsung Exit

Kalau container langsung exit setelah docker run, jalankan urutan ini:

- Step 1: docker ps -a
- Lihat semua container termasuk yang sudah exit — cek status
- Step 2: docker logs <nama-container>
- Baca log — ini sumber kebenaran utama
- Step 3: docker run --rm <image> ls /app
- Cek apakah file yang dibutuhkan masuk ke container
- Step 4: docker exec -it <nama> bash
- Masuk ke dalam container yang masih jalan untuk debug lanjut

Error Umum & Solusinya

Error	Penyebab	Solusi
Could not import module 'main'	main.py tidak ada di WORKDIR, atau paksa rm image	Cek apakah --rm image ls /app, sesuaikan CMD jadi app.main:app
port is already allocated	Port host sudah dipakai proses lain	Ganti -p 8001:8000 atau stop proses yang pakai port itu
ModuleNotFoundError	requirements.txt tidak ter-copy atau instalasi gagal	Cek perintah docker build — ada error di layer RUN pip install?
Connection refused	FastAPI listen ke 127.0.0.1 bukan 0.0.0.0	Tambahkan --host 0.0.0.0 di CMD uvicorn

7. Dockerfile Referensi — Risk Scoring API

Dockerfile yang digunakan hari ini untuk build Risk Scoring API:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Kenapa app.main:app bukan main:app? Karena main.py ada di dalam subfolder app/ (WEEK-8/app/main.py), bukan di root WEEK-8/. WORKDIR /app berarti working directory container adalah /app — dan dari sana, Python melihat main.py sebagai app.main, bukan main.

8. Command Referensi Lengkap

Command	Fungsi
docker build -t nama:tag .	Build image dari Dockerfile di direktori saat ini
docker run -d -p H:C --name N img	Run container: detached, port mapped, named

<code>docker ps</code>	Lihat container yang sedang berjalan
<code>docker ps -a</code>	Lihat semua container termasuk yang sudah exit
<code>docker logs nama</code>	Lihat log output container
<code>docker logs -f nama</code>	Follow / live stream log container
<code>docker exec -it nama bash</code>	Masuk ke dalam container yang sedang jalan
<code>docker stop nama</code>	Stop container
<code>docker rm nama</code>	Hapus container (harus sudah di-stop)
<code>docker images</code>	Lihat semua image di local
<code>docker rmi nama:tag</code>	Hapus image dari local